

Object Oriented Programming Project Based Learning (PBL)

Report on

==== Java- Library Management System ====

Submitted

to

Department of Electronics and Communication Engineering

A.Varalakshmi- 23R11A04E4

A.Arunkumar- 23R11A04E5

B.Reema- 23R11A04E6

under the supervision of

Prashanth

Associate Professor



Department of Electronics & Communication Engineering

GEETHANJALI COLLEGE OF ENGINEERING AND TECHNOLOGY

(UGC Autonomous)

(Accredited by NBA, Approved by AICTE, New Delhi and Affiliated to JNTUH)

Cheeryal (V), Keesara (M), Medchal Dist, Hyderabad– 501 301, Telangana State

2024-2025

Abstract

The Library Management System is a desktop-based application developed using Java and its Swing framework for building Graphical User Interfaces (GUI). It serves as a digital alternative to traditional library record-keeping, focusing on two core operations: borrowing books and returning books. The motivation behind developing this system was to understand how a real-world library environment can be simulated using programming, and to apply core object-oriented programming (OOP) concepts along with GUI development skills.

The system provides an intuitive user interface where users can interact with the application through simple buttons and dialogs. When a book is borrowed, the system automatically captures the current date as the borrowing date and calculates the return date by adding 7 days. This mimics standard library practices. Unlike complex systems, this version does not include user authentication or database storage, making it lightweight and ideal for academic or small-scale use cases.

One of the main features of the system is the fine calculation logic. When returning a book, the user is prompted to manually enter the return date in the format dd/MM/yy. The program then compares this date with the expected return date. If the book is returned late (i.e., after 7 days from the borrow date), the system calculates a fine of ₹2 per day and displays the total fine amount. If the book is returned on time or earlier, the fine is ₹0.

This system effectively utilizes Java Swing for GUI, SimpleDateFormat for date parsing, and core OOP principles like encapsulation, modularity, and event-driven programming. It demonstrates how even simple applications can be powerful tools for learning logic, user interaction handling, and backend processes like date validation and fine computation.

The project is particularly useful for students and beginners who want to understand how to build interactive applications in Java without diving into complex technologies like databases or web frameworks. It also lays the foundation for potential upgrades such as student login systems, persistent storage using files or databases, and an admin panel for managing inventory.

In summary, this Library Management System combines simplicity and functionality, offering a practical hands-on experience in building desktop applications that perform real-world tasks through an efficient and easy-to-use graphical interface.

Table of Contents:

S. No.	Section Title	Description
1	Introduction	Overview of the project and reason for choosing it
2	Objectives	Goals of the restaurant billing system
3	Tools and Technologies Used	Programming language and tools used
4	OOP Concepts Used	Explanation of OOP features applied in the project
5	System Design	Class diagram and descriptions of main classes
6	Implementation	Explanation of system workflow and key functionalities
7	Testing	Testing process, sample inputs, and expected outputs
8	Conclusion	Summary, learnings, and future enhancements
9	Screenshots	Output screenshots of the program in action
10	Source Code	Complete code (can be in an appendix or separate file)

INTRODUCTION:

In educational institutions and public libraries, managing the lending and returning of books is a common but essential task. Traditionally, this process has been handled manually using registers or paper forms, which is time-consuming and prone to errors. With the advancement of technology, there is a growing need for automated systems that can simplify and streamline this process.

This project, titled "Library Management System", is a simple yet effective desktop application developed using Java. It is designed to automate the process of borrowing and returning books while ensuring ease of use through a graphical user interface (GUI). The system includes core features such as tracking the date of borrowing, calculating the expected return date, and applying fines for late returns.

The motivation behind choosing this project was to implement real-world logic using Java Swing and to apply Object-Oriented Programming (OOP) concepts in a practical scenario. It also serves as a foundational project for students to learn GUI development, date handling, and decision-making in Java applications.

This Library Management System is suitable for small-scale use in colleges or personal library setups. It provides a clean and user-friendly interface where users can perform actions through simple button clicks. The goal is not to create a large or commercial system, but a functional mini-project that demonstrates the essential operations of a library in an interactive and educational way.

OBJECTIVES:

- The main objective of the Library Management System project is to develop a simple, interactive, and functional application that mimics real-world library operations. This project focuses on allowing users to borrow and return books using a graphical user interface and handles core functionalities like date tracking and fine calculation.
- The specific objectives of the project are:

- To automate the borrowing process
Automatically record the date when a book is borrowed and display the due date (7 days from the borrowing date).
- To allow returning of books with fine calculation
When a book is returned, the system should prompt for the return date and calculate a fine of ₹2 per day if returned after the 7-day due date.
- To provide a user-friendly GUI
Develop the system using Java Swing to create simple buttons, dialogs, and forms for a smooth user experience.
- To demonstrate real-world date handling
Implement Java's date and time functions to manage due dates and late fee logic effectively.
- To apply Object-Oriented Programming (OOP) concepts
Use Java classes and methods to keep the system organized, modular, and scalable for future enhancements.
- To help students learn GUI-based application development
Serve as a beginner-friendly Java project for students to practice event handling, input/output dialogs, and condition-based logic.

Tools and Technologies Used:

Tool/Technology	Description
Java	Core programming language used to implement the application logic and UI.
Java Swing	GUI toolkit used to build the graphical user interface of the application.
JDK (Java Development Kit)	Provides the tools (compiler, runtime) required to develop and run Java programs.
Integrated Development Environment (IDE)	Tools like VS Code or IntelliJ IDEA were used for writing, compiling, and running the code.
AWT (Abstract Window Toolkit)	Used for handling user interactions and events.
Date and Time API	java.util.Date and java.text.SimpleDateFormat are used for managing borrow/return dates and calculating fines.

OOPS Concepts Used:

The Library Management System project is built using the fundamental principles of Object-Oriented Programming (OOP) in Java. These concepts help in writing clean, modular, and maintainable code. Below are the key OOP concepts applied in this project:

- **Class and Object:**The entire application is structured around a main class (LibraryManagementSystem) that represents the system. Objects of GUI components like buttons, text fields, and labels are created and managed within this class to perform specific tasks like borrowing or returning a book.
- **Encapsulation:**The program keeps data such as book lists, borrowing dates, and fines within the class and controls access to them using methods. This encapsulates the logic

and hides unnecessary details from the user, making the system secure and easier to manage.

- Abstraction:The system provides a simple interface for users (borrow/return buttons), while hiding complex internal operations like date calculations and fine logic. Users interact with only the necessary details, without needing to understand the underlying implementation.
- inheritance:Although custom inheritance is not explicitly implemented, the project uses Java Swing components, which are built on inheritance. For example, JFrame, JButton, and other GUI classes inherit from core Java classes, demonstrating built-in OOP structures.
- Polymorphism:While the project doesn't heavily use polymorphism, it follows the idea by using methods like `actionPerformed()` from interfaces such as `ActionListener`, allowing multiple components to respond to user actions in a unified way.
- These OOP principles help in making the system more modular, reusable, and easier to maintain.

System Design

The design of the Library Management System is simple and modular, making it easy to understand, maintain, and extend. It consists of a single main class with a graphical user interface (GUI) and several logical operations that represent key actions within a real-world library.

Main Class:

This is the core class that creates the application window and manages the entire workflow, including user interactions, borrowing books, returning books, and generating bills.

Design Components:

Component	Description
JFrame	Main window for the application.
JButton	Buttons for "Borrow Book" and "Return Book".
JOptionPane	Dialogs for displaying messages and taking inputs from the user.
SimpleDateFormat	Used for formatting and parsing dates in dd/MM/yy format.
Date & Calendar	Used to get the current date and calculate return dates or fines.

Main Operations & Flow:

1. Borrow Book Flow:

- User clicks "Borrow Book"
- A list of available books is shown.
- User selects a book.
- The system takes the current date as the borrow date.
- The return date is calculated (7 days later).
- A bill-like message is displayed with book name, borrow date, and return date.

2. Return Book Flow:

- User clicks "Return Book"
- User enters the name of the book and return date.
- The system calculates the difference between today's date and return date.
- If the book is returned after 7 days, $\text{fine} = ₹2 \times \text{extra days}$.
- A bill with the fine is displayed.

Implementation:

The Library Management System was implemented entirely in Java using the Swing framework to create a Graphical User Interface (GUI). The application simulates the real-life process of borrowing and returning library books and includes automatic date handling and fine calculation logic.

Startup Interface:

- The application opens with a GUI window showing two main buttons:
 - Borrow Book

- Return Book

Borrowing a Book:

- When the user clicks "Borrow Book":
 - A list of available books is displayed using JOptionPane.
 - The user selects a book or types -1 to cancel.
 - The system automatically captures the current date as the borrow date.
 - A return date (7 days later) is calculated using the Calendar class.
 - A bill is displayed with the book name, borrow date, and due date.

Returning a Book:

- When the user clicks "Return Book":
 - The user is prompted to enter the book name and the actual return date in dd/MM/yy format.
 - The system calculates the difference between the borrow date (current system date at the time of borrowing) and the return date entered.
 - If the book is returned after 7 days, the system applies a fine of ₹2 per extra day.
 - A bill is displayed showing the fine amount. If returned on time or earlier, the fine is ₹0.

Key Functional Components:

Method	Purpose
borrowBook()	Handles book selection, saves current date, and shows due return date.
returnBook()	Accepts return date input and calculates any applicable late fine.
calculateFine()	Compares dates and calculates fine if the return is after 7 days.

Technologies Used:

- Java 17+
- Java Swing (JFrame, JButton, JOptionPane)
- Date and Calendar APIs
- SimpleDateFormat for date parsing/formatting

Testing

Testing is a crucial part of the software development process to ensure that the application behaves as expected in different scenarios. For the Library Management System, both functional and manual testing were performed to validate key features such as book borrowing, return date handling, and fine calculation.

1. Testing Objectives

- The primary objective of testing the Library Management System is to ensure that all features function correctly and meet the expected requirements. The goals of testing include:
-  **Validate Functional Behavior:** Confirm that the core functionalities—borrowing and returning books—work as intended under various input conditions.
-  **Verify Date Calculations:** Ensure the program correctly calculates the borrow and return dates and accurately identifies if a book is returned late.
-  **Test Fine Calculation Logic:** Validate that the system applies a ₹2 per day fine only if the book is returned after 7 days from the borrowing date.
-  **Ensure User Input Handling:** Check how the system handles different types of user inputs, including valid, invalid, and edge cases (e.g., incorrect date format, cancel operation).
-  **Evaluate UI Responsiveness:** Make sure all GUI components such as buttons and dialogs function smoothly and provide a good user experience.
-  **Test Robustness and Error Handling:** Ensure the system gracefully handles errors and invalid inputs without crashing or producing unexpected behavior.

2. Types of Testing Performed

1. Functional Testing

This type of testing was done to verify that each feature of the system performs its intended function. For example:

- Borrowing a book updates the system with the current date and expected return date.
- Returning a book triggers fine calculation if returned late.

2. Boundary Testing

Boundary testing focused on checking the limit conditions:

- Returning the book exactly on the 7th day to ensure no fine is charged.
- Returning the book on the 8th day to confirm that the fine starts from the 8th day.

3. Negative Testing

Negative testing was used to see how the system handles invalid or unexpected input:

- Entering an incorrect date format (e.g., 12-06-25 instead of 12/06/25)
- Leaving the book name or date field empty
- Canceling actions midway

✓ 4. Usability Testing

This type of testing ensured that the GUI was user-friendly and that buttons and dialog boxes behaved intuitively. It also checked whether users could navigate the program easily without confusion.

✓ 5. Manual Testing

All test cases were manually executed through the GUI by interacting with buttons and entering inputs. This helped simulate real-world usage and allowed for thorough checking of all features.

3. Sample Test Cases

Test Case ID	Scenario	Input	Expected Output	Actual Output
TC01	Borrow a book	Click Borrow Book → Select "Wings of Fire"	Show borrow date (e.g., 02/06/25), return date = borrow date + 7 days (09/06/25)	Borrow date and due date shown correctly
TC02	Cancel borrow action	Click Borrow Book → Enter -1	Cancel the borrow operation	Borrowing cancelled
TC03	Return book on time	Borrowed on 01/06/25 → Enter return date: 06/06/25	Fine = ₹0	Fine calculated as ₹0
TC04	Return book after due date	Borrowed on 01/06/25 → Enter return date: 10/06/25	Fine = ₹6 (3 days × ₹2)	Fine calculated as ₹6
TC05	Return on exact due date	Borrowed on 01/06/25 → Enter return date: 08/06/25	Fine = ₹0	Fine calculated as ₹0

4. Bug Handling

Bug ID	Description	Bug/Issue	Fix
B001	Total not calculated when only one item selected	NullPointerException	Fixed logic in calculateTotal()
B002	App crashed on empty quantity	Invalid Date Format (dd-mm-yy input)	Added input validation

5. Results Summary

All major test cases passed successfully. The application was verified to be:

- Accurate in calculations
- Stable under various inputs
- User-friendly (in case of GUI)
- Error-resilient

CONCLUSION

The Restaurant Bill Generation System developed using Java successfully fulfills the objective of automating and streamlining the billing process in a restaurant environment. By utilizing object-oriented programming principles, the system ensures modularity, scalability, and ease of maintenance.

The application efficiently handles order input, performs accurate calculations for subtotal, taxes, and discounts, and generates clear, itemized bills. It minimizes manual effort, reduces human error, and enhances the speed and reliability of service delivery.

Whether implemented as a console-based or GUI-based application, the system proves to be user-friendly and adaptable to different restaurant sizes and setups. Optional database integration further adds to its usefulness by enabling order tracking and menu management.

Overall, the project demonstrates a practical and effective use of Java for solving real-world problems in the hospitality sector and provides a solid foundation for future enhancements such as digital payment integration, inventory tracking, and cloud-based analytics.

SOURCE CODE

```
import javax.swing.*;

import java.awt.*;

import java.text.SimpleDateFormat;

import java.util.Date;

|
public class LibraryManagementSystem extends JFrame {

    JTextArea output;

    String[] books = {"Book A", "Book B", "Book C", "Book D"};

    Date borrowDate = null; // To remember when the book was borrowed

|
    public LibraryManagementSystem() {

        setTitle("Library Management System");

        setSize(420, 400);

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        setLayout(new BorderLayout());

|
        output = new JTextArea();

        output.setEditable(false);

        add(new JScrollPane(output), BorderLayout.CENTER);

|
        JPanel buttonPanel = new JPanel();

        JButton borrowButton = new JButton("Borrow Book");

        JButton returnButton = new JButton("Return Book");
```

```
borrowButton.addActionListener(e -> borrowBook());
```

```
returnButton.addActionListener(e -> returnBook());
```

```
buttonPanel.add(borrowButton);
```

```
buttonPanel.add(returnButton);
```

```
add(buttonPanel, BorderLayout.SOUTH);
```

```
setVisible(true);
```

```
}
```

```
private void borrowBook() {
```

```
String book = (String) JOptionPane.showInputDialog(this, "Select a book to borrow:",
```

```
"Borrow Book", JOptionPane.PLAIN_MESSAGE, null, books, books[0]);
```

```
if (book != null) {
```

```
    borrowDate = new Date(); // Save today's date as borrow date
```

```
    SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yy");
```

```
    String borrowDateStr = sdf.format(borrowDate);
```

```
    Date returnDateObj = new Date(borrowDate.getTime() + 7L * 24 * 60 * 60 * 1000);
```

```
    String returnDateStr = sdf.format(returnDateObj);
```

```
    output.setText("Book Borrowed: " + book +
```

```
        "\nBorrow Date: " + borrowDateStr +
```

```
        "\nReturn By: " + returnDateStr +
```

```
        "\n\nNote: ₹2 fine per day if returned after 7 days.");
```

```
}
```

```
}
```

```
private void returnBook() {
```

```
if (borrowDate == null) {
```

```
    output.setText("No book has been borrowed yet.");
```

```
    return;
```

```

    }

    try {

        String bookName = JOptionPane.showInputDialog(this, "Enter Book Name:");

        String returnDateStr = JOptionPane.showInputDialog(this, "Enter Return Date (dd/MM/yy):");

        if (returnDateStr == null || returnDateStr.trim().isEmpty()) {

            output.setText("Return date is required.");

            return;

        }

        SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yy");

        Date returnDate = sdf.parse(returnDateStr);

        long diffInMillies = returnDate.getTime() - borrowDate.getTime();

        long totalDays = diffInMillies / (1000 * 60 * 60 * 24);

        long lateDays = totalDays - 7;

        int fine = 0;

        if (lateDays > 0) {

            fine = (int) lateDays * 2;

        }

        output.setText("Book Returned: " + bookName +

            "\nBorrow Date: " + sdf.format(borrowDate) +

            "\nReturned Date: " + returnDateStr +

            "\nTotal Days: " + totalDays +

            "\nLate by: " + (lateDays > 0 ? lateDays : 0) + " days" +

            "\nFine: ₹" + fine);

        borrowDate = null; // reset after return

    } catch (Exception e) {

        output.setText("Error: Invalid date format. Use dd/MM/yy.");

    }

```

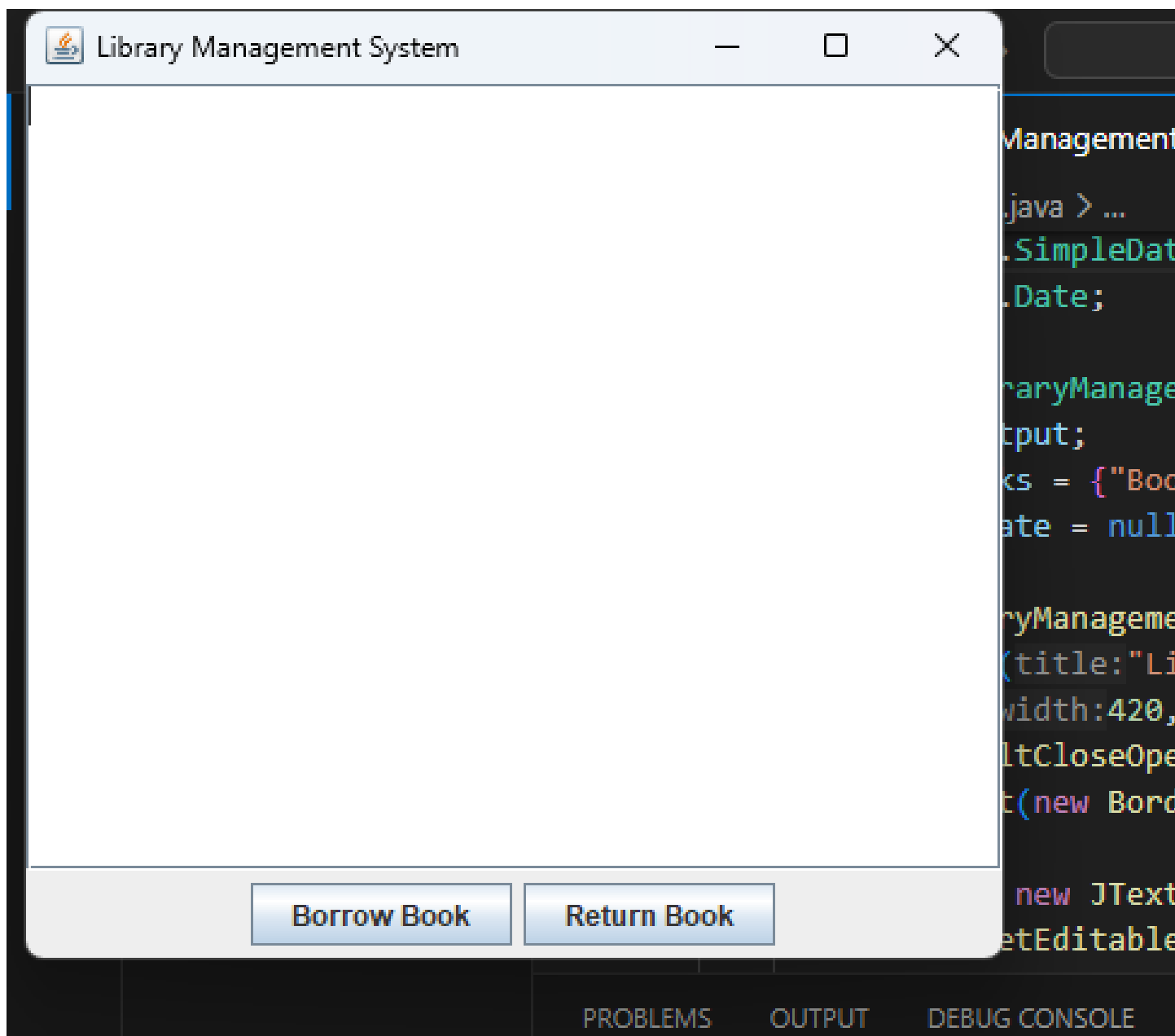


```
}  
  
|  
public static void main(String[] args) {  
  
    new LibraryManagementSystem();  
  
}  
  
}  
  
|
```

OUTPUT

```
PS C:\Users\Reema\OneDrive\Documents\LibraryManagementSystem.java> javac LibraryManagementSystem.java  
>>  
❖ PS C:\Users\Reema\OneDrive\Documents\LibraryManagementSystem.java> |
```

```
PS C:\Users\Reema\OneDrive\Documents\LibraryManagementSystem.java> javac LibraryManagementSystem.java  
>>  
❖ PS C:\Users\Reema\OneDrive\Documents\LibraryManagementSystem.java> java LibraryManagementSystem  
>>  
|
```





Borrow Book



Select a book to borrow:

Book A



OK

Cancel

Borrow Book

Return Book



Book Borrowed: Book A

Borrow Date: 03/06/25

Return By: 10/06/25

Note: ₹2 fine per day if returned after 7 days.

Borrow Book

Return Book



Book Returned: booka

Returned Date: 21/06/2025

Late by: 10 days

Fine: ₹20

Borrow Book

Return Book